4. What is the difference between CHAR and VARCHAR2 data types?
CHAR: Fixed-length storage. If the defined length is not fully used, it is padded with spaces.
VARCHAR2: Variable-length storage. Only the actual data is stored, saving space when the full length is not needed.

16. What is the purpose of the GROUP BY clause?
The GROUP BY clause is used to arrange identical data into groups. It is typically used with aggregate functions (such as COUNT, SUM, AVG) to perform calculations on each group rather than on the entire dataset.

23. What are the differences between SQL and NoSQL databases?
SQL Databases:
Use structured tables with rows and columns.
Rely on a fixed schema.s
Offer ACID properties.
NoSQL Databases:
Use flexible, schema-less structures (e.g., key-value pairs, document stores).
Are designed for horizontal scaling.
Often focus on performance and scalability over strict consistency.

27. What is a trigger in SQL?
A trigger is a set of SQL statements that automatically execute in response to certain

events on a table, such as INSERT, UPDATE, or DELETE. Triggers help maintain data consistency, enforce business rules, and implement complex integrity constraints.

28. What is the purpose of the SQL SELECT statement?
The SELECT statement retrieves data from one or more tables. It is the most commonly used command in SQL, allowing users to filter, sort, and display data based on specific criteria.

33. What is a composite primary key?
A composite primary key is a primary key made up of two or more columns.

34. How is data integrity maintained in SQL databases?
Data integrity refers to the accuracy, consistency, and reliability of the data stored in the database. SQL databases maintain data integrity through several mechanisms:

Constraints: Ensuring that certain conditions are always met. For example, NOT NULL ensures a column cannot have missing values, FOREIGN KEY ensures a valid relationship between tables, and UNIQUE ensures no duplicate values.
Transactions: Ensuring that a series of operations either all succeed or all fail, preserving data consistency.

Triggers: Automatically enforcing rules or validations before or after changes to data.
Normalization: Organizing data into multiple related tables to minimize redundancy and prevent anomalies.
These measures collectively ensure that the data remains reliable and meaningful over time

38. How does the CASE statement work in SQL?
The CASE statement is SQL's way of implementing conditional logic in queries. It evaluates conditions and returns a value based on the first condition that evaluates to true. If no condition is met, it can return a default value using the ELSE clause.

Example:

```
SELECT ID,
       CASE
           WHEN Salary > 100000 THEN 'High'
           WHEN Salary BETWEEN 50000 AND 100000
THEN 'Medium'
           ELSE 'Low'
       END AS SalaryLevel
FROM Employees;
```

40. What is the purpose of the COALESCE function?
The COALESCE function returns the first non-NULL value from a list of expressions. It's commonly used to provide default values or handle missing

data gracefully.

Example:
SELECT COALESCE(NULL, NULL, 'Default Value') AS Result;

example 2 : SELECT COALESCE(NULL, NULL, 'Hello', 'World') AS result;
//hello

41. What are the differences between SQL's COUNT() and SUM() functions?

1. COUNT(): Counts the number of rows or non-NULL values in a column.

Example:

SELECT COUNT(*) FROM Orders;

2. SUM(): Adds up all numeric values in a column.

Example:

SELECT SUM(TotalAmount) FROM Orders;

42. What is the difference between the NVL and NVL2 functions?

NVL(): Replaces a NULL value with a specified replacement value.

If expression is NULL, it returns replacement_value.
If expression is not NULL, it returns expression itself.

NVL2(): Evaluates two values:
If the first argument is NOT NULL, returns the second argument.
If the first argument is NULL, returns the third

argument.
Example:

```
SELECT NVL(Salary, 0) AS AdjustedSalary
FROM Employees;
```

43. How does the RANK() function differ from DENSE_RANK()? read page 14 for example

RANK(): Assigns a rank to each row, with gaps if there are ties. Assigns a rank to each row based on the specified order.
If there is a tie (same value), all tied rows get the same rank, but the next rank skips numbers.

DENSE_RANK(): Assigns consecutive ranks without any gaps. Assigns consecutive ranks without skipping numbers.
If there is a tie, all tied rows get the same rank, but the next rank follows sequentially

Example:

```
SELECT Name, Salary, RANK() OVER (ORDER BY Salary
DESC) AS Rank
FROM Employees;
```

If two employees have the same salary, they get the same rank, but RANK() will skip a number for the next rank, while DENSE_RANK() will not.

44. What is the difference between ROW_NUMBER() and RANK()?

ROW_NUMBER(): Assigns a unique number to each row regardless of ties.
RANK(): Assigns the same number to tied rows and leaves gaps for subsequent ranks.
Example:

```
SELECT Name, ROW_NUMBER() OVER (ORDER BY Salary
DESC) AS RowNum
FROM Employees;
```

45. What are common table expressions (CTEs) in SQL?

A CTE is a temporary result set defined within a query. It improves query readability and can be referenced multiple times.

here query can be select ,insert ,delete etc.

Example:

```
WITH TopSalaries AS (
    SELECT Name, Salary
    FROM Employees
    WHERE Salary > 50000
)
SELECT * FROM TopSalaries WHERE Name LIKE 'A%';
```

46. What are window functions, and how are they used?

read page 14 for example

Window functions perform calculations across a set of rows that are related to the current row. Unlike aggregate functions, they don't collapse the result set.

Example: Calculating a running total

```
SELECT Name, Salary, SUM(Salary) OVER (ORDER BY
Salary) AS RunningTotal
```

```
FROM Employees;
```

46. What are window functions, and how are they used?
Window functions perform calculations across a set of rows that are related to the current row. Unlike aggregate functions, they don't collapse the result set.

Example: Calculating a running total

```
SELECT Name, Salary, SUM(Salary) OVER (ORDER BY Salary) AS RunningTotal
FROM Employees;
```

49. What are the trade-offs of using indexes in SQL databases?
Advantages

Faster query performance, especially for SELECT queries with WHERE clauses, JOIN conditions, or ORDER BY clauses.
Improved sorting and filtering efficiency.
Disadvantages:

Increased storage space for the index structures.
Additional overhead for write operations (INSERT, UPDATE, DELETE), as indexes must be updated whenever the underlying data changes.
Potentially slower bulk data loads or batch

inserts due to the need to maintain index integrity.
In short, indexes make read operations faster but can slow down write operations and increase storage requirements.

51. What are temporary tables, and how are they used?
Temporary tables are tables that exist only for the duration of a session or a transaction. They are useful for storing intermediate results, simplifying complex queries, or performing operations on subsets of data without modifying the main tables.

1. Local Temporary Tables:

Prefixed with # (e.g., #TempTable).
Only visible to the session that created them.
Automatically dropped when the session ends.
2. Global Temporary Tables:

Prefixed with ## (e.g., ##GlobalTempTable).
Visible to all sessions.
Dropped when all sessions that reference them are closed.
Example:

```
CREATE TABLE #TempResults (ID INT, Value
VARCHAR(50));
```

```
INSERT INTO #TempResults VALUES (1, 'Test');
SELECT * FROM #TempResults;
```

52. What is a materialized view, and how does it differ from a standard view?

Standard View:

A virtual table defined by a query.

Does not store data; the underlying query is executed each time the view is referenced.

A standard view shows real-time data.

Materialized View:

A physical table that stores the result of the query.

Data is precomputed and stored, making reads faster.

Requires periodic refreshes to keep data up to date.

materialized view is used to store aggregated sales data, updated nightly, for fast reporting.

54. What are the advantages of using sequences over identity columns?

1. Greater Flexibility:

Can specify start values, increments, and maximum values.

Can be easily reused for multiple tables.

2. Dynamic Adjustment: Can alter the sequence without modifying the table structure.

3. Cross-Table Consistency: Use a single sequence for multiple related tables to ensure unique identifiers across them.
In short, sequences offer more control and reusability than identity columns.

55. How do constraints improve database integrity?
Constraints enforce rules that the data must follow, preventing invalid or inconsistent data from being entered:

NOT NULL: Ensures that a column cannot contain NULL values.
UNIQUE: Ensures that all values in a column are distinct.
PRIMARY KEY: Combines NOT NULL and UNIQUE, guaranteeing that each row is uniquely identifiable.
FOREIGN KEY: Ensures referential integrity by requiring values in one table to match primary key values in another.
CHECK: Validates that values meet specific criteria (e.g., CHECK (Salary > 0)).
By automatically enforcing these rules, constraints maintain data reliability and consistency.

56. What is the difference between a local and a

global temporary table?
Local Temporary Table:
Prefixed with # (e.g., #TempTable).
Exists only within the session that created it.
Automatically dropped when the session ends.
Global Temporary Table:
Prefixed with ## (e.g., ##GlobalTempTable).
Visible to all sessions.
Dropped only when all sessions referencing it are closed.
Example:

```
CREATE TABLE #LocalTemp (ID INT);
CREATE TABLE ##GlobalTemp (ID INT);
```

57. What is the purpose of the SQL MERGE statement?
The MERGE statement combines multiple operations INSERT, UPDATE, and DELETE into one. It is used to synchronize two tables by:

Inserting rows that don’t exist in the target table.
Updating rows that already exist.
Deleting rows from the target table based on conditions
Example:

```
MERGE INTO TargetTable T
```

```
USING SourceTable S
ON T.ID = S.ID
WHEN MATCHED THEN
    UPDATE SET T.Value = S.Value
WHEN NOT MATCHED THEN
    INSERT (ID, Value) VALUES (S.ID, S.Value);
```

58. How can you handle duplicates in a query without using DISTINCT?

1. GROUP BY: Aggregate rows to eliminate duplicates

```
SELECT Column1, MAX(Column2)
FROM TableName
GROUP BY Column1;
```

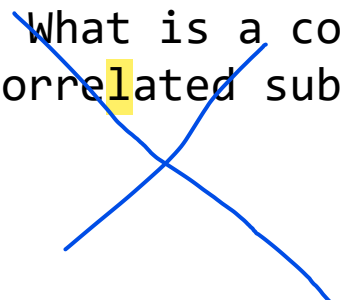
2. ROW_NUMBER(): Assign a unique number to each row and filter by that

```
WITH CTE AS (
    SELECT Column1, Column2, ROW_NUMBER() OVER
(PARTITION BY Column1 ORDER BY Column2) AS RowNum

    FROM TableName
)
SELECT * FROM CTE WHERE RowNum = 1;
```

59. What is a correlated subquery?

A correlated subquery is a subquery that



references columns from the outer query. It is re-executed for each row processed by the outer query. This makes it more dynamic, but potentially less efficient.

Example:

```
SELECT Name,
       (SELECT COUNT(*)
        FROM Orders
        WHERE Orders.CustomerID =
Customers.CustomerID) AS OrderCount
FROM Customers;
```

43.) Example Using RANK()

```
SELECT Name, Salary, RANK() OVER (ORDER BY Salary DESC) AS Rank
FROM Employees;
```

Output:

```
Name Salary Rank
Alice 50000 1
Bob   45000 2
Eve   45000 2
John 40000 4
```

Example Using DENSE_RANK():

```
SELECT Name, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRank
FROM Employees;
```

Output:

```
Name Salary DenseRank
Alice 50000 1
Bob   45000 2
Eve   45000 2
John 40000 3
```

46.) syntax:

```
SELECT column_name,
    window_function() OVER (
        PARTITION BY column_name
        ORDER BY column_name
    ) AS result
FROM table_name;
```

Example 1: Running Total (SUM() OVER())

```
SELECT Name, Salary, SUM(Salary) OVER (ORDER BY Salary) AS RunningTotal
FROM Employees;
```

```
Name   Salary  RunningTotal
Alice  3000   3000
Bob    4000   7000
Eve    5000   12000
John   6000   18000
```